ASUS TUF-AX3000 V2: Hardware-Analyse & Firmware-Recovery

Low-Level De-Bricking via UART, U-Boot & SPI-Programming

STATUS: BRICKED // INITIATING RECOVERY PROTOCOL...

NotebookLM

Das chirurgische Protokoll: 4 Phasen der Wiederherstellung

01

Diagnose & Anatomie

Identifkation von SPI-Flash & NAND-Storage auf dem PCB

02

Toolchain & Instrumentarium

CH341A, Spannungswandler & Testklammern

03

Umgebung & Zugang

UART-Kommunikation, PuTTY-Parameter & TFTP-Server

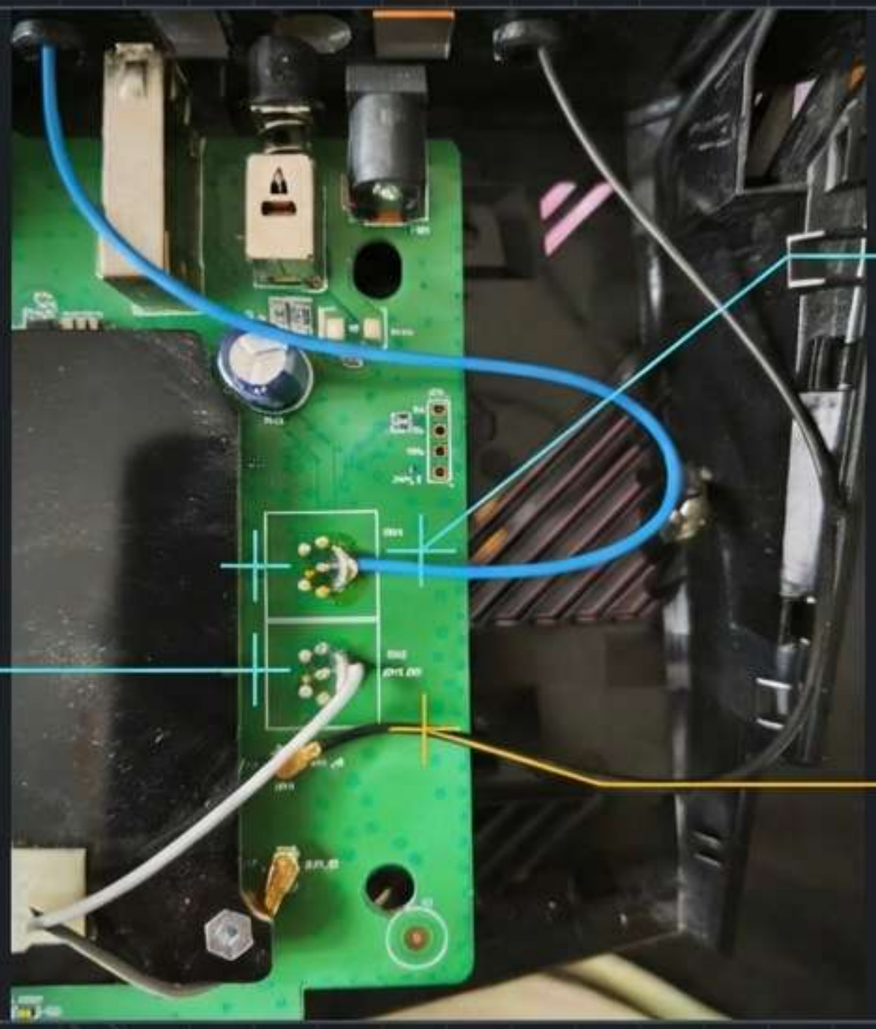
04

Operation & Log-Analyse

U-Boot Interaktion, flash_img_upgrade & UBI Volume Rebuild

NotebookLM

Platinen-Topologie & Serieller Direktzugang (Layer 1)



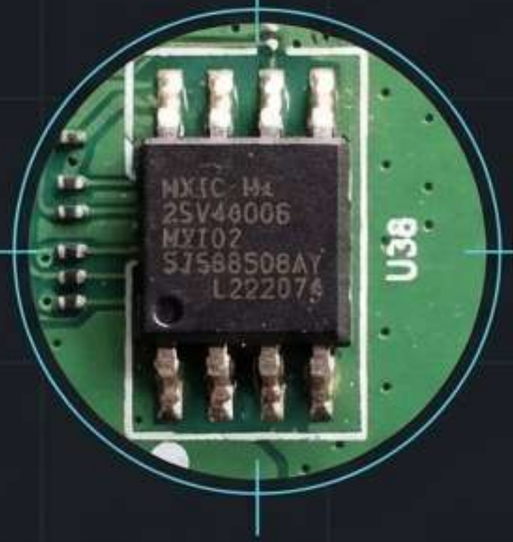
UART Pinout-Legende

- RX (Receive) -> Pin 1
- TX (Transmit) -> Pin 2
- GND (Ground) -> Pin 3
- VCC (3.3V) -> Nicht belegt

⚠ CRITICAL WARNING: Isolierte Stromversorgung sicherstellen. VCC-Pin des Routers niemals mit dem UART-Adapter verbinden, um zerstörerische Spannungs-Schleifen zu vermeiden. Spannung erfolgt über separates Router-Netzteil.

NotebookLM

Speichermodul-Architektur: Bootloader vs. Main Storage

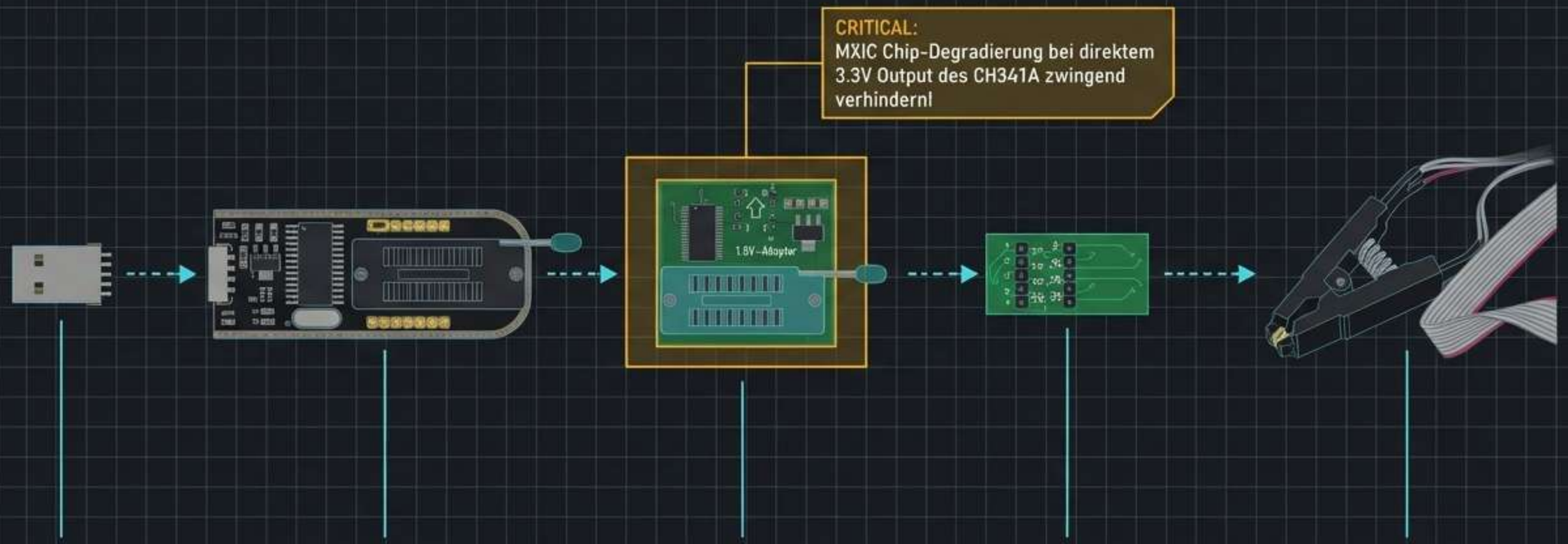


MXIC MX25V4006E	
Rolle	SPI-Flash (Beinhaltet den kritischen Bootloader/U-Boot)
Logik-Spannung	1.8V / 3.3V kompatibel
Tooling-Bedarf	CH341A + SOP8 Testklammer + 1.8V Level-Shifter

Micron 2WG47 (NW874)	
Rolle	Hauptspeicher / NAND-Flash (Beinhaltet ASUS WRT Firmware & UBI Volumes)
Kapazität	High-Density Storage
Tooling-Bedarf	Indirekter Flash via TFTP & RAM (Kein direkter Clip-Zugriff empfohlen)

NotebookLM

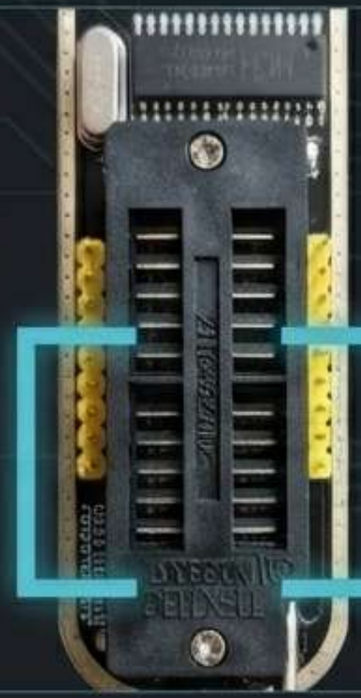
Die Programmer-Toolchain (Signalfluss-Architektur)



1. Host Interface (PC USB)
2. Base Programmer (CH341A)
3. Voltage Level Shifter (1.8V-Adapter)
4. Breakout Board (Pin-Routing)
5. Target Interface (SOP8 Testklammer)

NotebookLM

CH341A ZIF-Sockel Mapping



JetBrains Mono (DIN Pro)

Aufdruck-Analyse: Die Platine trennt strikt zwischen 25 SPI BIOS (hintere Reihe) und 24 I2C EEPROM (vordere Reihe).

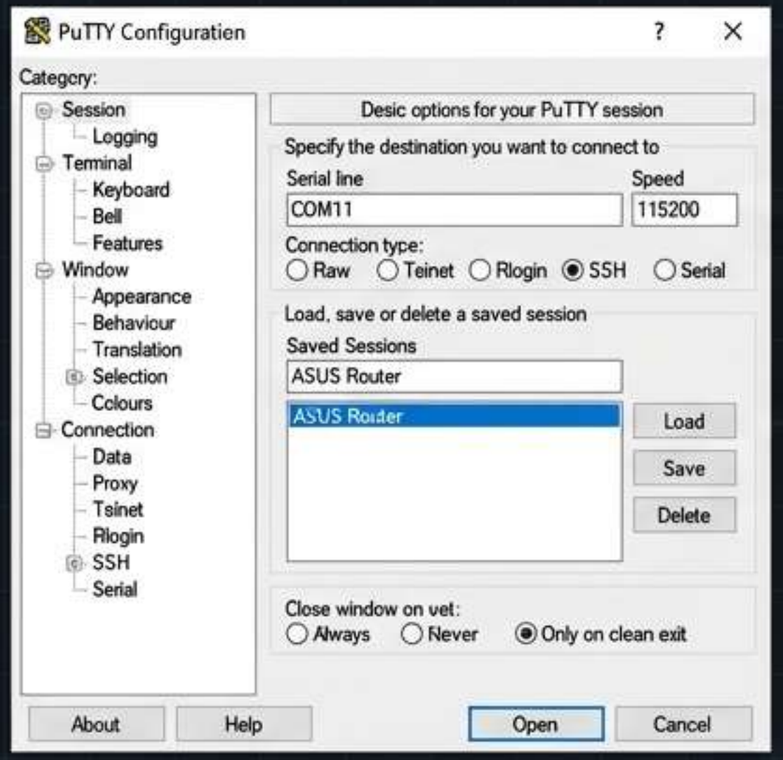
JetBrains Mono (DIN Pro)

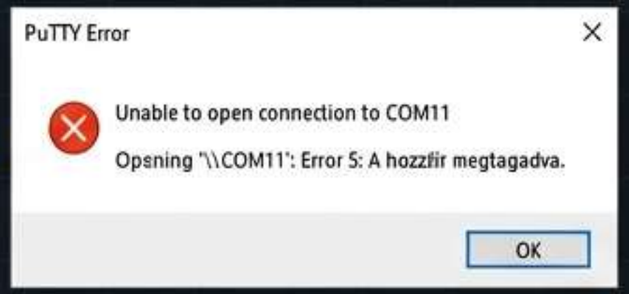
Execution: Für den MXIC MX25V4006E (SPI Flash) muss zwingend die 25XX Pin-Konfiguration nahe dem Hebel genutzt werden. Pin 1 muss korrekt auf die Markierung ausgerichtet sein.

NotebookLM

UART-Environment & Der Terminal-Gatekeeper

DIN Pro





PARAMETER-SETUP

Line: COM11
Speed: 115200 Baud (Kritische Frequenz für ASUS-Bootloader)
Connection Type: Serial

ERROR ANALYSIS: ERROR 5

Symptom: Unable to open connection to COM11. Error 5: A hozzáférés megtagadva.

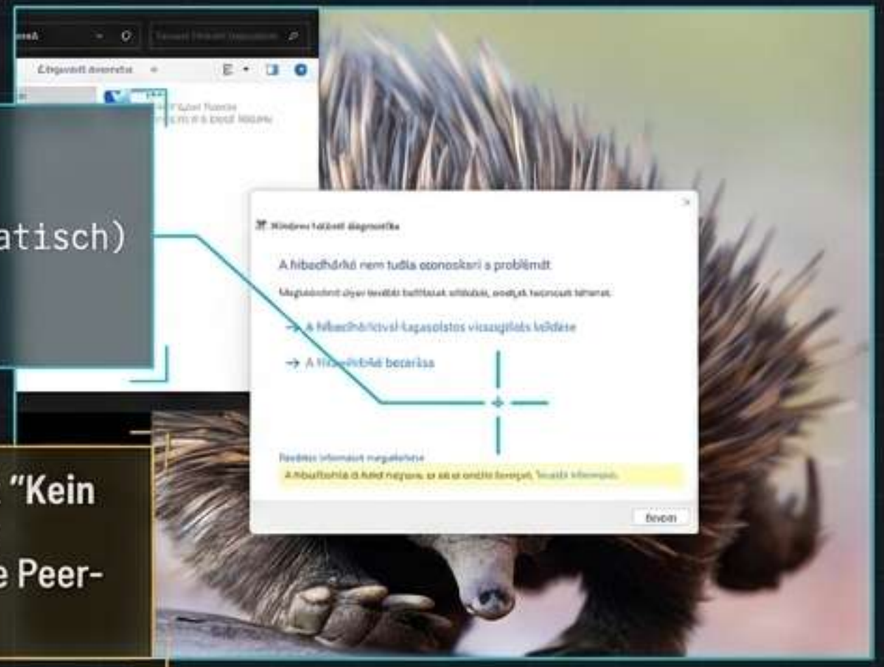
Root Cause: COM-Port wird bereits von einem anderen Prozess blockiert oder CH340-Treiber ist im Konflikt-Status.

Resolution: Task-Manager Bereinigung / Re-Enumeration des USB-Ports erzwingen.

NotebookLM

Subnetz-Isolierung & TFTP-Vorbereitung

```
Target-IP (Router):     192.168.1.1
Host-IP (TFTP Server):  192.168.1.10 (Statisch)
Subnetzmaske:           255.255.255.0
```



HINWEIS: Windows-Netzwerkdiagnose meldet oft "Kein Internet" (siehe Screenshot rechts) – dies ist der korrekte und gewünschte Status für eine isolierte Peer-to-Peer Flashing-Sitzung.

NotebookLM

Bootloader Interrogation (printenv & help)

`boardid=TUF_AX3000_V2`

DIN Pro: Verifikation der exakten Hardware-ID. Verhindert das Flashen eines inkompatiblen Firmware-Images.

`bootcmd=printenv;run once;sdk boot_img`

Die primäre Boot-Sequenz, die aktuell aufgrund des Bricks fehlschlägt.

`ipaddr=192.168.1.1`

Bestätigt die Ziel-Adresse für den bevorstehenden TFTP-Transfer.

NotebookLM

Payload Delivery: Der TFTP-Handshake

```
> setenv serverip 192.168.1.10
> tftpboot asus.pkgtb
```

```
Filename 'asus.pkgtb'.
Load address: 0x1000000
Loading:. tftp_start :. Set timout handler and timeout:5000
tftp_send: tftp_filename is asus.pkgtb

chk cf magic: edfe0dd0, comfw_id=9

.Check name in fdt: TUF-AX3000_V2(13), rt_buildname=TUF-AX30
00_V2, chk result=0

-- end of tftp transfer --
tftp_complete

         4.7 MiB/s
done
Bytes transferred = 53822164 (33542d4 hex)
=>
```

Telemetry Data

- Target File: asus.pkgtb
- Load Address: 0x1000000
- Transfer Rate: 4.7 MiB/s
- Bytes Transferred: 53822164
- Magic Number Check: edfe0dd0

VERIFIZIERUNG:
Image-Integrität bestanden

NotebookLM

Die flash_img_upgrade Anatomie (UBI Volume Rebuild)

=> sdk flash_img_upgrade -s asus.pkgtb

```
kmd% image
 img:     Image index for flashing/booting. Valid values: [1
 ]. Default index is determined from metadata
 boardid: Boardid to determine boot configuration. If omitte
d default configuration node is used
 conf:    Configuration name used for flashing img bundle. I
d omitted default configuration node is used

httpd_start
 - without arguments, start httpd server
otp set <row> <val> fuses <val> at <row> to OTP . Warning! o
tp fuse operations are irreversible
otp get <row>        reads content at <row> from  OTP
otp secure           Aohtong!!!!! This command will turn boar
d to NFO boot secure mode.
                     On success the board will be halted and
must be power cycled.
rering_firmp         Configures proper settings for an image
that was written in raw from .

|-> flash_img_upgrade -s asus.pkgtb
Unknown command 'flash_img_upgrade' - try 'help'
=> sdk  flash_img_upgrade -s asus.pkgtb
A0 Checking hashes) for FIT Image at 81000000 ...
   Hashes) for Image 0 (bootfs_6756_a0r): sha256+
   Hashes) for Image 1 (nand_squashfs): sha256+
Flashing bootfs (bootfs_6756_a0r: 0x9a7724 bytes from 0x100
500) to bootfs1.....
Remove UBI volume bootfs1 (id 3)
Creating static volume bootfs1 of size 11173668
Flashing rootfs (nand_squashfs: 0x29ac000 bytes from 0x19a6
ds) to rootfs1.....
Remove UBI volume rootfs1 (id 4)
Creating dynamic volume rootfs1 of size 44744704
45684120 bytes written to volume rootfs1
INFO: Loader not found in configuration conf_6756_a0+_nand_
squashfs ...skipping!
INFO: Image upgrade successfull!!
INFO: Committing Image!!
Read 2048 bytes from volume metadata1 to 1de71e60
read from metadata1 returned 0
Getting committed 2 valid 1,2 seq 7,8
Remove UBI volume metadata1 (id 1)
Creating static volume metadata1 of size 4736
Remove UBI volume metadata2 (id 2)
Creating static volume metadata2 of size 4736
Setting committed 1 valid 1,2 seq 9,8
=>
```

Schritt 1: Hash Validation
sha256+ Check für bootfs und rootfs.

Schritt 2: Volume Teardown
Remove UBI volume bootfs1 (id 3) & rootfs1 (id 4).
Zerstört das korrumpierte Dateisystem restlos.

Schritt 3: Volume Creation
Aufbau neuer statischer (11173668 bytes) und dynamischer (44744704 bytes) Volumes.

Schritt 4: Squashfs Flashing
Das eigentliche Schreiben des rootfs (0x29ac000 bytes).

Schritt 5: Commit Sequence
Setzen der Metadata-Validierungs-Flags (seq 7,8 -> seq 9,8).
Image upgrade successfull!!

NotebookLM

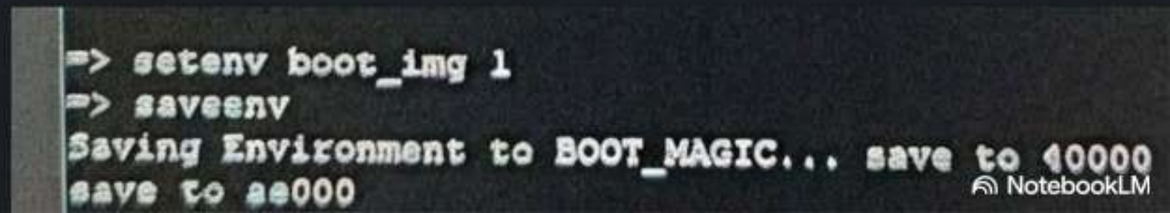
Boot-Umgebung sichern (The Final Commit)

```
=> setenv boot_img 1
```

Setzt den Zeiger auf das soeben geflashte, intakte Image-Volume.

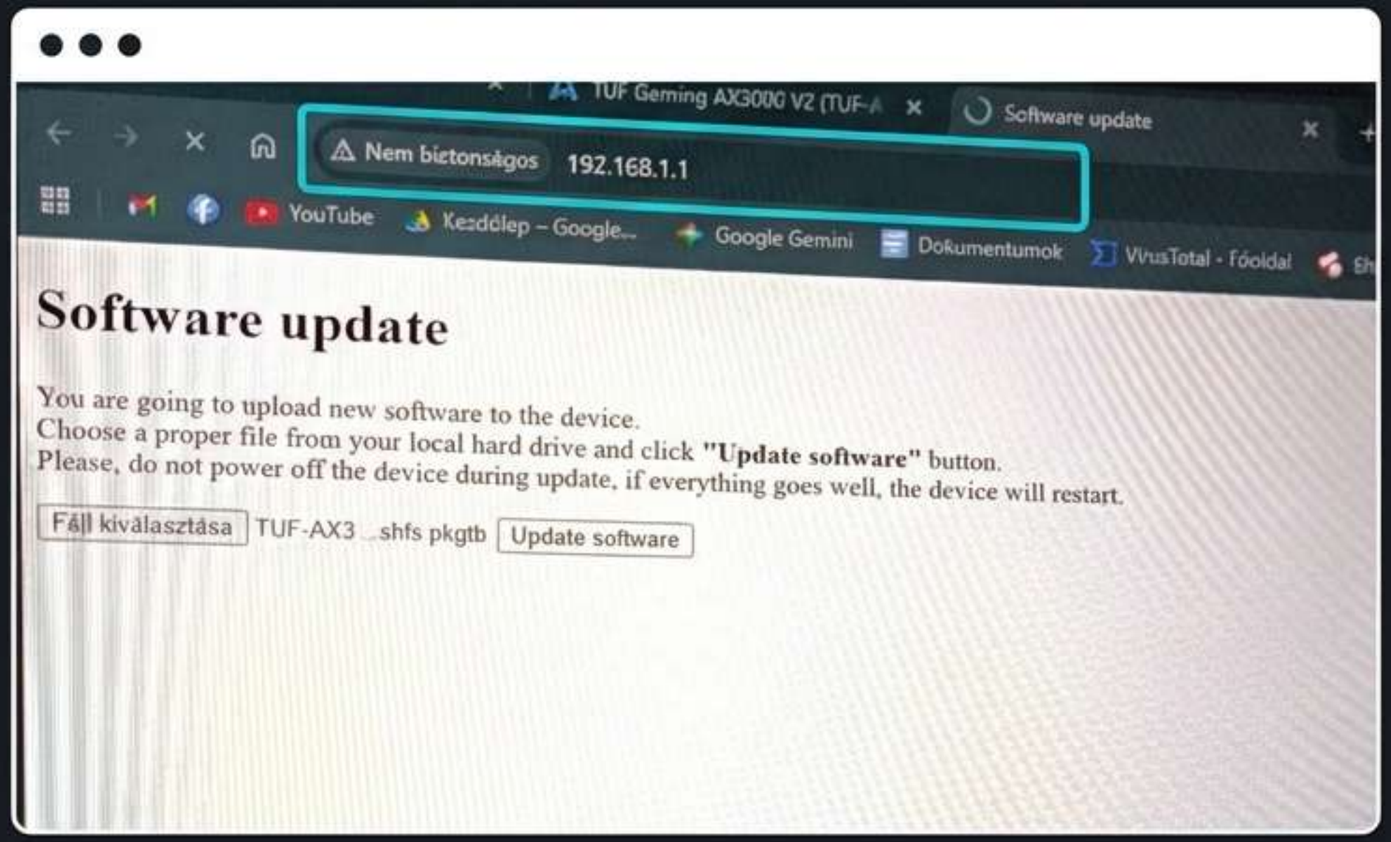
```
=> saveenv
Saving Environment to BOOT_MAGIC...
save to 40000
```

Schreibt die Änderungen permanent in den SPI-Flash. Ohne diesen Schritt wäre der Router nach dem Reboot weiterhin im Bricked-Status.



NotebookLM

System-Boot & Verifizierung der Recovery-GUI



HTTP-Server: ONLINE	**Fazit:** Der Bootloader lädt wieder erfolgreich den Kernel. Das System ist aus dem Hard-Brick befreit und bereit für ein sauberes, reguläres Firmware-Update via Web-Interface.
GUI-Modus: Emergency Firmware Upload / Software Update (ASUS Rescue Mode)	

NotebookLM

Mission Debriefing: 3 Key Takeaways für ASUS TUF Recovery

Baudrate ist Gesetz

Exakt `115200` Baud. Jede Abweichung erzeugt unlesbaren Garbage-Output im UART-Terminal.

Spannungs-Disziplin (1.8V)

Der MXIC-Flashchip (MX25V4006E) überlebt keine 3.3V-Logik auf Dauer. Der 1.8V-Level-Shifter am CH341A ist nicht optional, sondern Pflicht.

UBI-Architektur respektieren

Ein einfaches `cp` reicht nicht. Das `flash_img_upgrade` Script muss genutzt werden, da es dynamisch UBI-Volumes zerstört und anhand von Checksummen neu anlegt.

NotebookLM